

后端第 6 题：Linux 的使用

学号：2026090902009

姓名：吴雨泽

题目：Linux 的使用

Task 1：虚拟机、Linux 发行版与网络

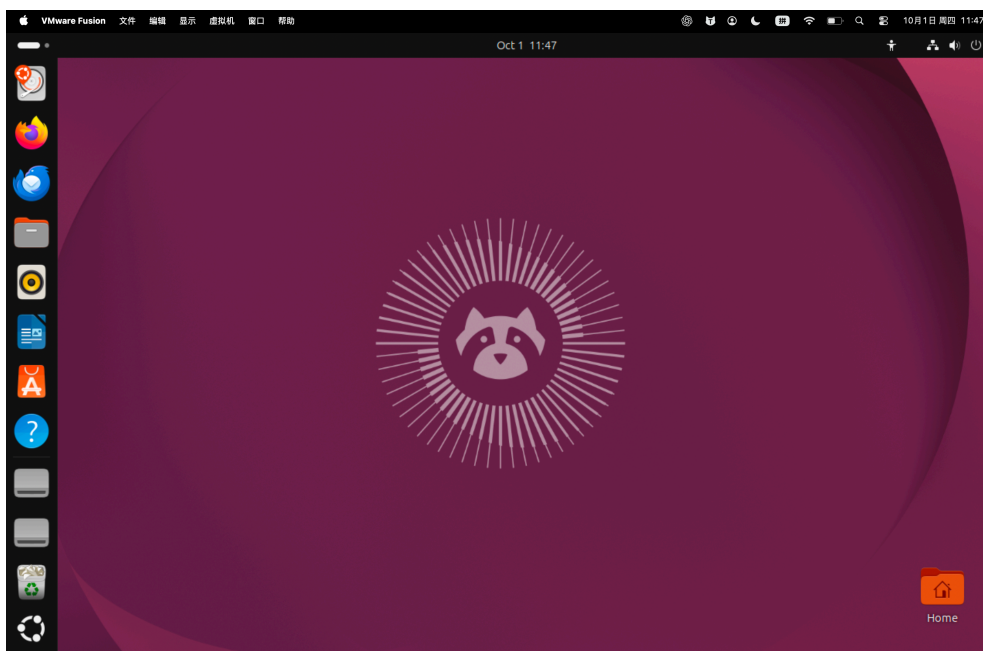
1. 安装环境

我的电脑使用 macOS，虚拟机软件是 VMware Fusion。本次启动的是 Ubuntu 26.04.1 LTS，架构为 aarch64 (ARM64)，当前用户为 ubuntu。由于使用 Apple Silicon，镜像也选择 ARM64 版本，与题目中的 Windows + Workstation + amd64 环境有所不同。

检查根文件系统时，`findmnt /` 显示 `overlay` 和 `/cow`，说明目前仍是镜像的试用环境，还没有完成虚拟硬盘安装。题目指定的 Ubuntu 24.04 安装要求仍需要另外落实，不能用这次试用桌面代替。

ISO 镜像可以理解为安装光盘的文件版本。创建虚拟机时加载镜像，分配处理器、内存和硬盘空间，再按照 Ubuntu 安装程序完成设置。安装好以后，系统写入虚拟硬盘，日常启动不需要一直挂着安装镜像。

下面是之前保存的 Ubuntu 桌面截图。



2. 对 Linux 发行版的理解

Linux 本身主要指内核，负责管理进程、内存、设备等。平时安装的 Ubuntu 则是一套完整的发行版：除了内核，还带有命令行工具、软件包管理器、桌面环境和软件仓库。可以把内核理解成共同的基础，不同发行版在这个基础上选择了不同的软件和维护方式。

发行版	特点和用途
Ubuntu	基于 Debian，资料比较多，桌面使用方便，适合开始学习 Linux。
Debian	重视稳定性，稳定版的软件更新比较谨慎，适合希望环境长期保持一致的场景。
Fedora	比较积极地引入新技术，适合接触较新的 Linux 软件和功能。
RHEL	面向企业，重视长期维护、支持和兼容性。
Arch Linux	采用滚动更新，配置自由度高，但需要自己处理更多安装和维护工作。
deepin	比较重视桌面界面和日常使用体验，也是常见的国产发行版。

对我目前的学习需求来说，Ubuntu 的教程和软件安装方式更容易上手。学习时先把目录、命令和权限弄清楚，比频繁更换发行版更有帮助。

3. 三种网络连接方式

方式	虚拟机怎样连接网络	适合的情况
NAT	虚拟机位于虚拟网络中，通常借用宿主机的网络连接访问外网。	下载软件、上网和日常实验。外部设备主动连接虚拟机时，往往需要端口转发。
桥接	虚拟机通过宿主机的网卡接入实际局域网，通常获得局域网里的独立地址。	希望同一局域网的其他设备直接访问虚拟机。能否访问还取决于网络和防火墙设置。
仅主机	通常只连接宿主机和同一个虚拟网络中的虚拟机，不直接访问外网。	宿主机与虚拟机之间做隔离实验。

我对这三种方式的区分是：NAT 方便虚拟机向外访问；桥接方便把虚拟机作为局域网中的一台独立机器；仅主机则把通信范围留在宿主机和虚拟机之间。NAT 下宿主机一般也能访问虚拟机，不一定要为了远程连接就切成桥接。

虚拟机适合练习，是因为误操作通常被限制在虚拟系统里。不过共享文件夹和网络仍可能连接宿主机，所以也不能把它当成完全没有风险的环境。

Task 2：目录结构和常用命令

1. Linux 的目录结构

Windows 常用 C 盘、D 盘区分文件位置，Linux 则从 `/` 开始组织整棵目录树。其他硬盘、分区也可以挂载到这棵树的某个目录下。

目录	主要作用
/	根目录，整棵目录树的起点。
/home	普通用户的主目录通常放在这里，例如 /home/用户名。
/root	root 用户的主目录。它与根目录 / 是两个不同的位置。
/etc	系统和服务的配置文件。
/usr	大量程序、库和共享资源；不是普通用户的个人文件夹。
/var	日志、缓存等经常变化的数据。
/tmp	临时文件，里面的数据可能被自动清理。
/bin、/sbin	基本命令和系统管理命令。现代 Ubuntu 中可能链接到 /usr 下。
/lib	程序和系统需要的基础库等，也可能链接到 /usr/lib。
/dev	设备文件，如磁盘和终端对应的设备节点。
/proc、/sys	系统提供的虚拟接口，用来查看进程、内核和设备信息。
/boot	内核和启动相关文件。
/mnt、/media	常用于挂载文件系统或可移动存储设备。

这里有三个容易混淆的写法：/ 是根目录，~ 是当前用户的主目录，. 是当前所在目录。.. 则表示上一级目录。

2. 创建题目要求的目录

题目要求在主目录中建立下面的结构：

```
~/linux-practice/
├─ notes/
├─ scripts/
└─ test-files/
```

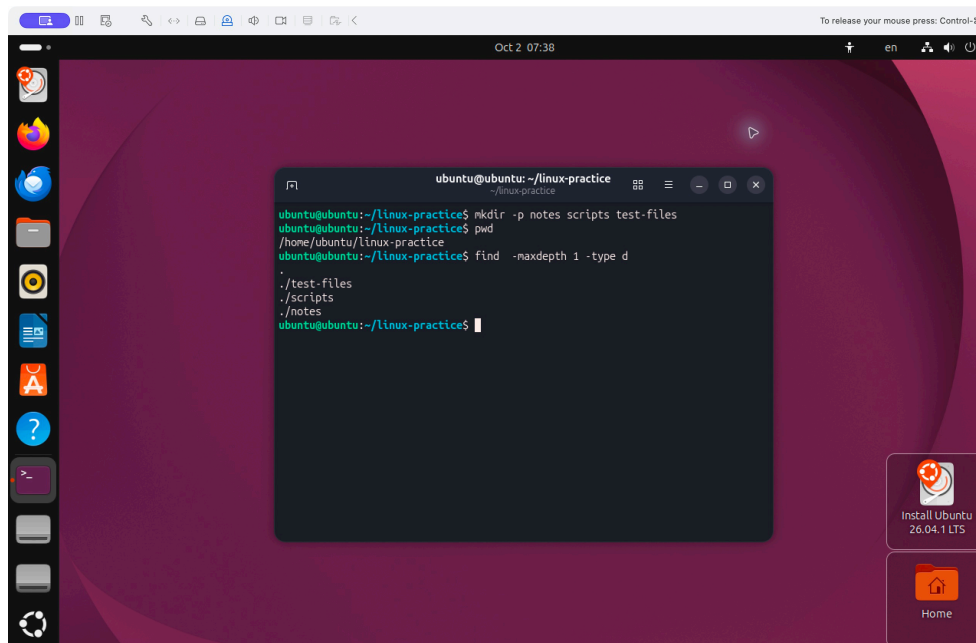
在 Ubuntu 终端中执行：

```
mkdir -p ~/linux-practice/notes ~/linux-practice/scripts ~/linux-practice/test-files
cd ~/linux-practice
find . -maxdepth 1 -type d | sort
```

mkdir 用来创建目录，-p 可以同时创建缺少的上级目录；目录已经存在时也不会仅因此报错。cd 进入目录，find 则用于查看刚才创建的结构。sort 只是让显示顺序更方便核对。

touch 用来创建空文件；如果文件已经存在，它会更新时间，不会自动清空内容。目录和文件的用途不同，所以不能用 touch notes 来代替 mkdir notes。

实际创建了 notes、scripts、test-files 三个目录，下面是 find 显示的结果。



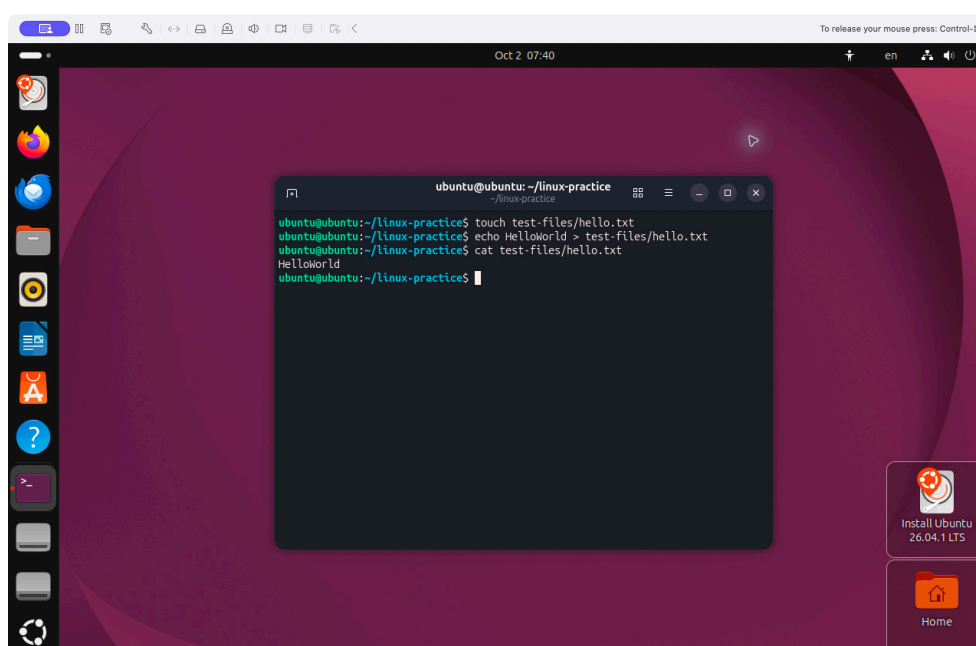
3. 写入并查看 HelloWorld

```
cd ~/linux-practice
touch test-files/hello.txt
echo HelloWorld > test-files/hello.txt
cat test-files/hello.txt
```

其中 `>` 把输出写入文件，会覆盖原来的内容；`>>` 则是追加到文件末尾。`echo` 默认在输出末尾加上换行，`cat` 用来显示文件内容。这里即使省略 `touch`，重定向也能创建文件，但把两个步骤分开更容易看清楚过程。

预期 `cat` 输出：

```
HelloWorld
```



4. 常用命令速查表

命令示例	用途
<code>pwd</code>	查看当前所在目录。
<code>ls -la</code>	查看文件详情，包括隐藏文件。
<code>cd ..</code>	进入上一级目录。
<code>cp hello.txt hello-copy.txt</code>	复制文件。
<code>mv hello-copy.txt backup.txt</code>	移动文件或重命名。
<code>rm backup.txt</code>	删除文件，执行前要确认路径。
<code>less /var/log/syslog</code>	分页查看日志，按 <code>q</code> 退出。文件是否存在取决于日志配置。
<code>head -n 10 file.txt</code>	查看前十行。
<code>tail -f file.log</code>	持续查看文件末尾新增的内容，按 <code>Ctrl+C</code> 停止。
<code>find . -name '*.txt'</code>	从当前目录查找文本文件。
<code>grep -n 'Hello' hello.txt</code>	查找包含 Hello 的行，并显示行号。
<code>ps aux</code> 、 <code>top</code>	查看进程及资源占用。
<code>df -h</code>	查看文件系统空间。
<code>du -sh ~/linux-practice</code>	统计练习目录占用的空间。
<code>ip addr</code> 、 <code>ip route</code>	查看接口地址和路由。
<code>ss -lnt</code>	查看正在监听的 TCP 端口。
<code>curl -I https://www.glimmer.org.cn</code>	请求网站响应头，需要已安装 curl。
<code>man chmod</code> 、 <code>chmod --help</code>	查看命令说明。

命令通常按“命令名 + 选项 + 参数”的形式使用。比如 `head -n 10 file.txt` 中，`head` 是命令，`-n 10` 指定行数，`file.txt` 是要处理的文件。

Task 3: Vim 文本编辑器

1. 编辑 hello.txt

Vim 最需要先记住的是模式切换。普通模式下按键通常表示操作，插入模式下才能输入正文；按 `Esc` 回到普通模式，再输入 `:` 可以执行保存、退出等命令。

为了保留 Task 2 中 `test-files/hello.txt` 的内容，这个练习在 `linux-practice` 目录中新建另一个 `hello.txt`：

```
cd ~/linux-practice
vim hello.txt
```

如果提示没有安装 Vim，在 Ubuntu 中安装：

```
sudo apt update
sudo apt install vim
```

进入 Vim 后按下面的顺序操作：

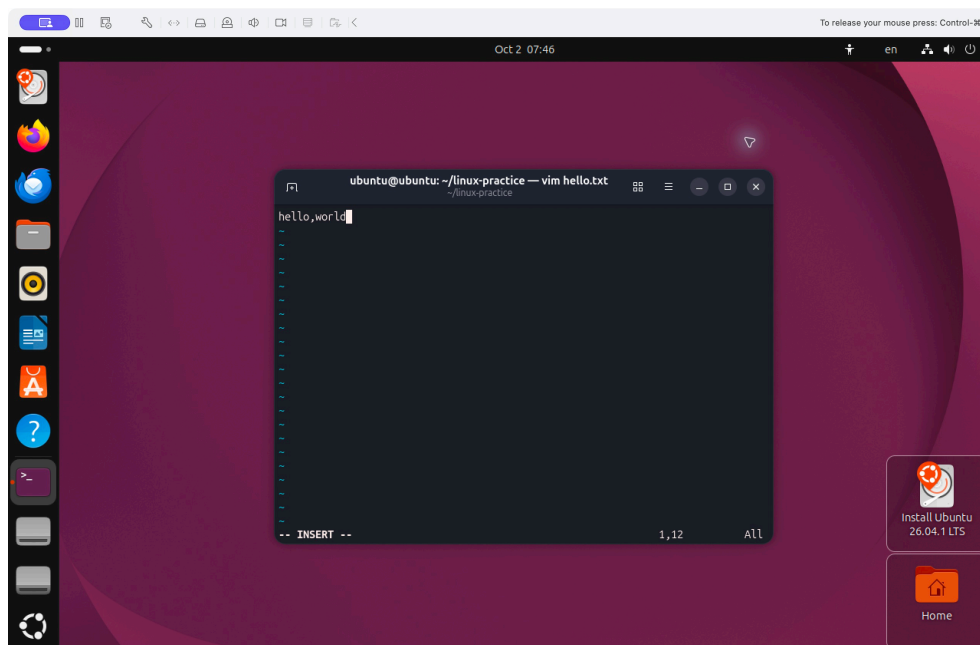
1. 按 `i` 进入插入模式。
2. 输入 `hello,world`。
3. 按 `Esc` 返回普通模式。
4. 输入 `:wq`，按回车保存并退出。
5. 回到终端，执行 `cat hello.txt` 检查内容。

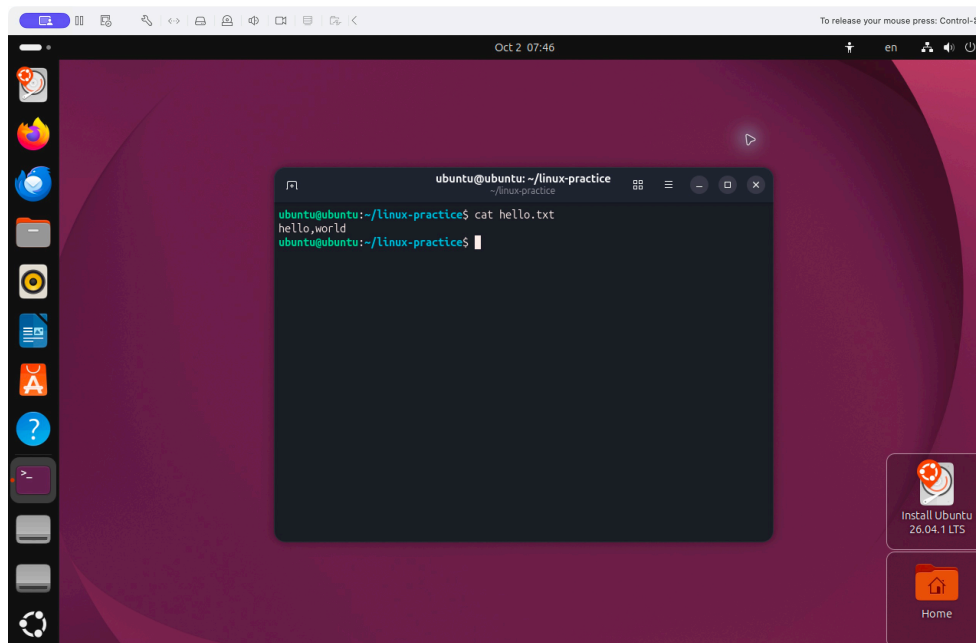
预期输出：

```
hello,world
```

打开一个不存在的路径时，Vim 会准备一个新缓冲区；真正执行保存后，文件才会写入磁盘。这也解释了为什么只打开 Vim、直接放弃退出，不一定会留下新文件。

下面是实际进入插入模式后的内容，以及保存退出后用 `cat` 检查的结果。





2. Vim 简易速查表

下面的按键除特别注明外，均从普通模式开始使用。

按键或命令	作用
i、a	在光标前、光标后进入插入模式。
I、A	在本行第一个非空白字符前、行尾进入插入模式。
o、O	在当前行下方、上方新建一行并进入插入模式。
Esc	退出插入模式，返回普通模式。
h、j、k、l	向左、下、上、右移动。
gg、G	跳到文件开头、末尾。
0、\$	跳到行首、行尾。
dd	删除当前行。
yy、p	复制当前行，将内容粘贴到光标之后；复制整行时粘贴到下一行。
u、Ctrl+r	撤销、重做。
/关键词	向后搜索，回车确认；n 查下一个，N 反向查找。
:w	保存。
:q	退出；有未保存修改时通常会提示。
:wq	保存并退出。
:q!	放弃未保存的修改并退出。
:set number	显示行号。

刚开始不必一次记住所有快捷键。能熟练切换模式、移动、搜索、保存退出，就已经可以修改一般配置文件了。

Task 4: IP、端口和远程连接

1. 对 IP 和端口的理解

IP 地址用来标识网络中的接口，通信时需要它来找到目标。端口则用于区分这台机器上的不同网络服务。例如，同一台服务器可以用 22 端口提供 SSH，用 80 端口提供 HTTP，用 443 端口提供 HTTPS。

“门牌号和房间号”的比喻比较好记，但实际连接还要区分 TCP、UDP 等协议。例如 TCP 的 80 端口和 UDP 的 80 端口不是同一个通信端点。

IPv4 地址有 32 位，常见写法是四个十进制数字，例如 192.168.1.20；IPv6 地址有 128 位，常见写法由冒号分隔。端口号范围是 0~65535，0~1023 属于系统端口范围，但不意味着每一个都正在被某个程序使用。开发服务时通常选择未被占用的较高端口，例如 8080。

2. 查看虚拟机地址并准备 SSH

在 Ubuntu 虚拟机里执行：

```
ip -4 addr
ip route
```

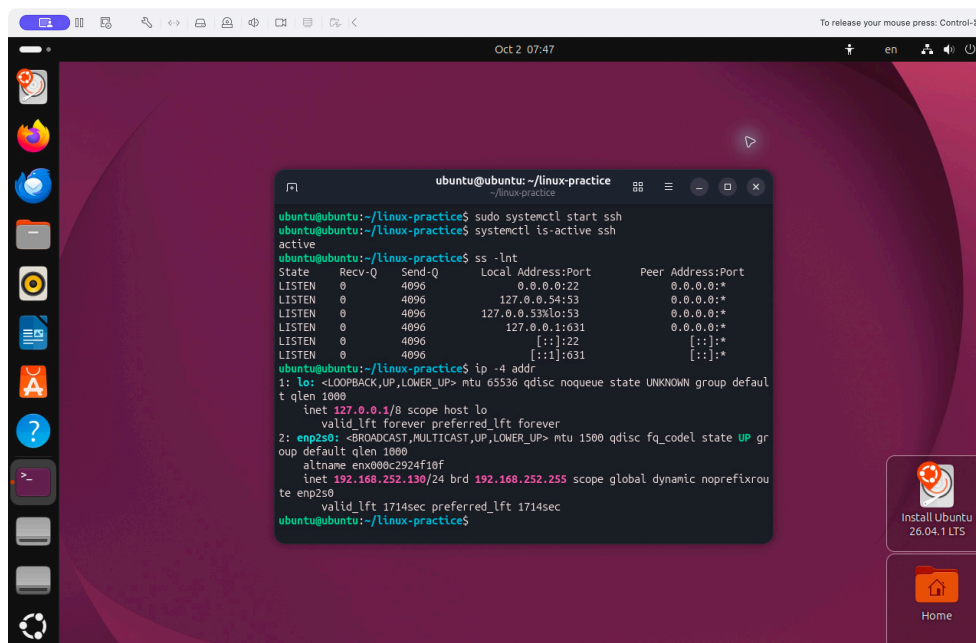
查看实际网卡下的 inet 地址。网卡名称不一定是 ens33，地址也不一定以 192.168 开头。127.0.0.1 是回环地址，表示当前机器自己，不能直接把它填成宿主机访问虚拟机的目标地址。地址后面的 /24 等内容表示网络前缀长度，填主机地址时只取 IP 部分。

在 Ubuntu 中安装并启用 SSH 服务：

```
sudo apt update
sudo apt install openssh-server
sudo systemctl enable --now ssh
systemctl status ssh --no-pager
ss -lnt
```

如果 SSH 使用默认配置，检查时应能看到相应的监听信息；Ubuntu 的具体版本也可能采用 socket 激活，必要时查看 `systemctl status ssh.socket --no-pager`。如果 UFW 已启用且阻止 SSH，可按实验网络的需要配置允许规则，不需要直接关闭整个防火墙。

本次实际查到的虚拟机地址为 192.168.252.130。启动 SSH 后，`systemctl is-active ssh` 返回 active，`ss -lnt` 显示 22 端口正在监听。



3. FinalShell 的连接设置

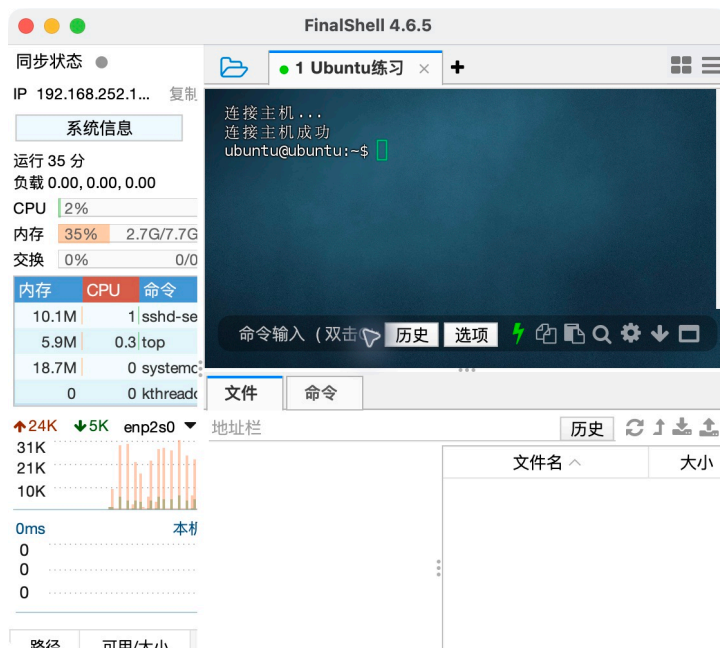
在 Mac 上打开 FinalShell，新建一个 SSH 连接：

项目	填写内容
名称	例如 Ubuntu练习，用于自己区分连接。
主机	192.168.252.130，本次虚拟机实际 IP。
端口	默认配置下填写 22。
用户名	ubuntu，当前试用环境的普通用户。
认证	根据自己的配置使用密码或密钥。

第一次连接时核对服务器身份信息，再完成认证。连接成功后，在远程终端中执行 `whoami`、`hostname` 和 `pwd`，确认当前操作的是 Ubuntu 虚拟机。也可以在 Mac 终端用 `ssh 用户名@虚拟机IP` 做同样的连接测试，其中用户名和 IP 要替换成实际值。

远程连接方便的地方是：可以直接在宿主机里管理虚拟机，不需要一直切换到虚拟机桌面；服务器没有图形界面时，也能通过终端修改配置、查看日志和运行程序。FinalShell 还把终端和文件管理放在同一个界面中，传文件会更直观。

FinalShell 实际连接后，终端显示“连接主机成功”，并出现 `ubuntu@ubuntu:~$` 提示符。左侧能够读取虚拟机的 IP、运行时间和资源占用，说明 SSH 登录已经完成。



Task 5: 用户、用户组与权限

1. root 和用户组

root 是 Linux 的超级用户，通常 UID 为 0。它有权限完成系统管理操作，例如修改系统配置、安装软件和管理其他用户。普通用户可以通过 `sudo`，在获得授权并完成认证后执行需要管理员权限的命令。

用户组可以把需要相同权限的人组织在一起。比如几个人共同维护练习资料，可以把他们加入同一个组，再把相关文件设置为组成员可读或可写。

在 Ubuntu 中，新建用户和组可以用下面的命令。执行前应确认这些名字没有被使用：

```
sudo adduser learner
sudo groupadd study
sudo usermod -aG study learner
id learner
```

`adduser` 会交互式创建用户，`groupadd` 创建组。`usermod -aG` 把用户追加到附属组中，其中 `-a` 不能随便省略，否则可能覆盖已有的附属组列表。新组成员关系通常要重新登录后，才会体现在用户会话中。

2. 为什么不一直使用 root

root 的问题在于权限太大。路径写错、误删文件或运行了有问题的脚本，都可能影响整个系统。程序被攻击时，如果它本来就以 root 身份运行，攻击者能获得的权限也会更大。

因此，日常编辑文件、写程序和查看自己目录下的内容，用普通用户就够了。遇到确实需要管理员权限的操作，再针对那条命令使用 `sudo`。它不是让操作自动变安全，执行前仍然要知道命令会改什么。

3. 读懂 rwx

例如，`ls -l` 中出现：

```
-rw-r-----
```

第一位 `-` 表示普通文件；如果是 `d`，则表示目录。后面九位分成三组：所有者、所属组、其他用户。这个例子中，所有者可读可写，同组用户只可读，其他用户没有权限。

权限	普通文件	目录
<code>r</code>	读取文件内容。	列出目录中的名字。
<code>w</code>	修改文件内容。	通常配合 <code>x</code> ，在目录里创建、删除或重命名目录项。
<code>x</code>	允许把文件作为程序执行，还需要文件格式等条件满足。	进入目录，并按名称访问目录中的项目。

删除文件主要看父目录的权限，不是只看文件自身有没有 `w`。这点和“可写就是可以删除文件”的直觉不太一样。

4. 用 `chmod` 修改权限

用 Task 2 创建的文件进行练习：

```
cd ~/linux-practice
chmod 600 test-files/hello.txt
ls -l test-files/hello.txt
chmod 640 test-files/hello.txt
ls -l test-files/hello.txt
```

预期第一轮权限为 `-rw-----`，第二轮为 `-rw-r-----`。文件大小、用户名、组名和时间以实际显示为准。

数字权限把 `r`、`w`、`x` 分别记为 4、2、1，每一组把需要的权限相加：

数字	权限
7	<code>rwX</code> : 4 + 2 + 1
6	<code>rw-</code> : 4 + 2
5	<code>r-X</code> : 4 + 1
4	<code>r--</code>
0	<code>---</code>

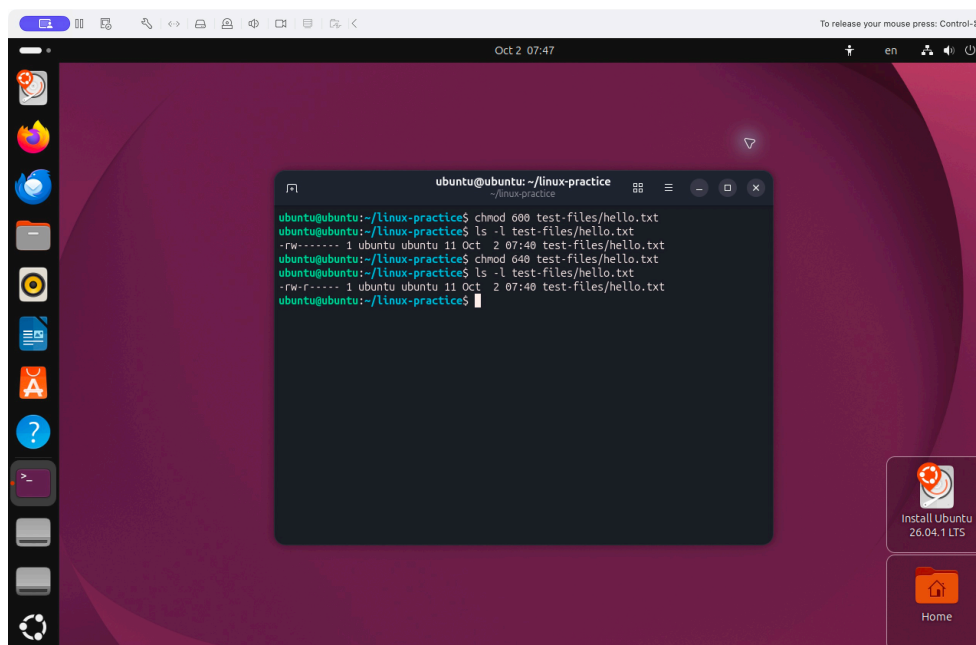
所以 `640` 表示所有者 6、所属组 4、其他用户 0。`755` 常用于需要执行的程序或需要被访问的目录，`600` 常用于仅允许本人读写的文件。

也可以按角色单独增减权限：

```
chmod u+x scripts/example.sh
chmod o-r test-files/hello.txt
chmod g+w test-files/hello.txt
```

上面的 `example.sh` 是语法示例，运行前需要先创建实际脚本。`u`、`g`、`o` 分别表示所有者、所属组和其他用户；`+` 增加权限，`-` 去掉权限。设置权限时应按用途分配，不必为了省事把文件全部改成 `777`。

实际执行后，权限从 `-rw-----` 变成了 `-rw-r-----`，与 `600`、`640` 的含义一致。



The screenshot shows an Ubuntu desktop with a purple background. A terminal window titled 'ubuntu@ubuntu: ~/linux-practice' is open, displaying the following commands and output:

```
ubuntu@ubuntu:~/linux-practice$ chmod 600 test-files/hello.txt
ubuntu@ubuntu:~/linux-practice$ ls -l test-files/hello.txt
-rw----- 1 ubuntu ubuntu 11 Oct 2 07:40 test-files/hello.txt
ubuntu@ubuntu:~/linux-practice$ chmod 640 test-files/hello.txt
ubuntu@ubuntu:~/linux-practice$ ls -l test-files/hello.txt
-rw-r----- 1 ubuntu ubuntu 11 Oct 2 07:40 test-files/hello.txt
ubuntu@ubuntu:~/linux-practice$
```

The desktop also shows a sidebar with application icons and two dock icons at the bottom right: 'Install Ubuntu 26.04.1 LTS' and 'Home'.

参考资料

- 本题要求：Glimmer 后端第 6 题
- Ubuntu 官方文档：OpenSSH server
- GNU Coreutils：chmod 用法
- Vim 的具体按键可在本机通过 `:help` 查看。